

Difference between Macro and Procedure

While both provides code reusability, their main differences are:

Macro: Uses the same memory location as where it is called, meaning the macro code is appended on that caller place. Macro is usually defined at the beginning of a program

Procedure: Stored in separate memory location. When called, the program jumps to that location to execute the instructions in that procedure.

Task 1 solution:

```
.MODEL SMALL
```

```
.STACK 100H
```

```
PRINT_STR MACRO STRING
```

```
    MOV AH,09H
```

```
    LEA DX, STRING
```

```
    INT 21H
```

```
ENDM
```

```
MACARRAY MACRO DIGIT_VAL, ARR_ADDR, ARR_LEN
```

```
    MOV AL, DIGIT_VAL
```

```
    LEA SI, ARR_ADDR
```

```
    MOV CX, ARR_LEN
```

```
    CALL SEARCH_PROC
```

```
ENDM
```

.DATA

ARRAY DB 2,0,4,7,1,9

ARR_LENGTH DW 6

INPUT_CHAR DB ?

MSG_INPUT DB 'INPUT: \$'

MSG_OUT_FOUND DB 0DH, 0AH, 'OUTPUT: DIGIT \$'

MSG_FOUND_SUFFIX DB ' FOUND\$'

MSG_OUT_NOT_FOUND DB 0DH, 0AH, 'OUTPUT: DIGIT \$'

MSG_NOT_FOUND_SUFFIX DB ' NOT FOUND\$'

.CODE

MAIN PROC

MOV AX, @DATA

MOV DS, AX

PRINT_STR MSG_INPUT

MOV AH, 01H

INT 21H

MOV INPUT_CHAR, AL

SUB AL, 30H

MACARRAY AL, ARRAY, ARR_LENGTH

MOV AH, 4CH

INT 21H

MAIN ENDP

SEARCH_PROC PROC

MOV BL, AL

SEARCH_LOOP:

MOV AL, [SI]

CMP AL, BL

JE FOUND

INC SI

LOOP SEARCH_LOOP

NOT_FOUND:

PRINT_STR MSG_OUT_NOT_FOUND

MOV AH, 02H

MOV DL, INPUT_CHAR

INT 21H

PRINT_STR MSG_NOT_FOUND_SUFFIX

RET

FOUND:

PRINT_STR MSG_OUT_FOUND

MOV AH, 02H

MOV DL, INPUT_CHAR

INT 21H

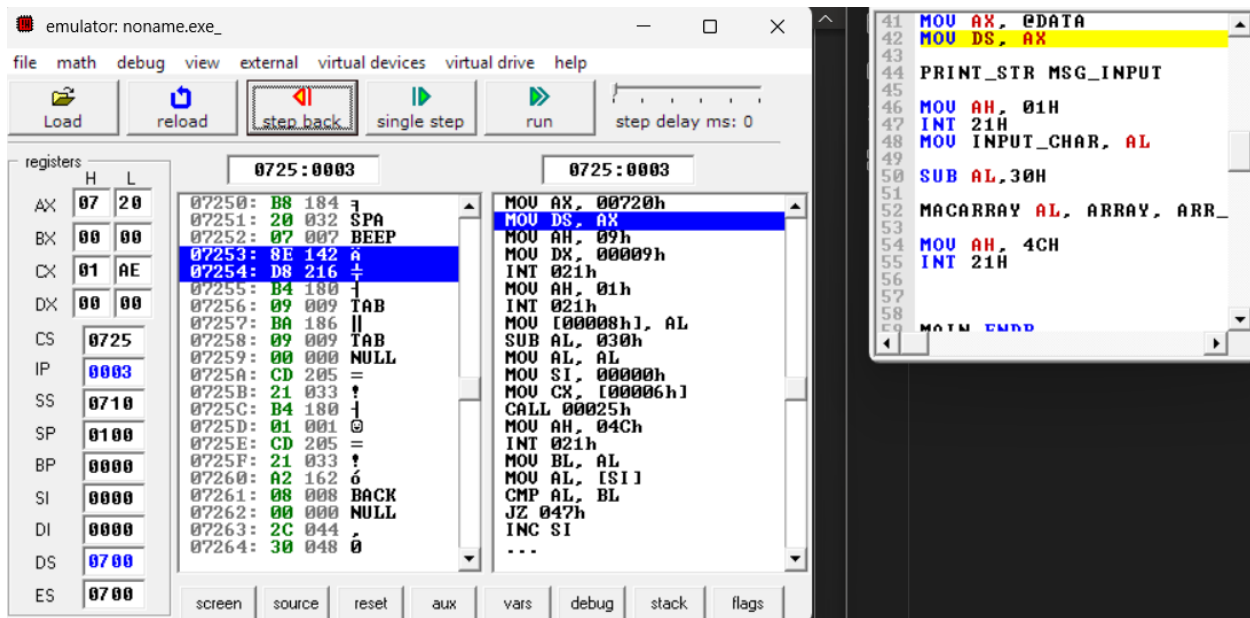
PRINT_STR MSG_FOUND_SUFFIX

RET

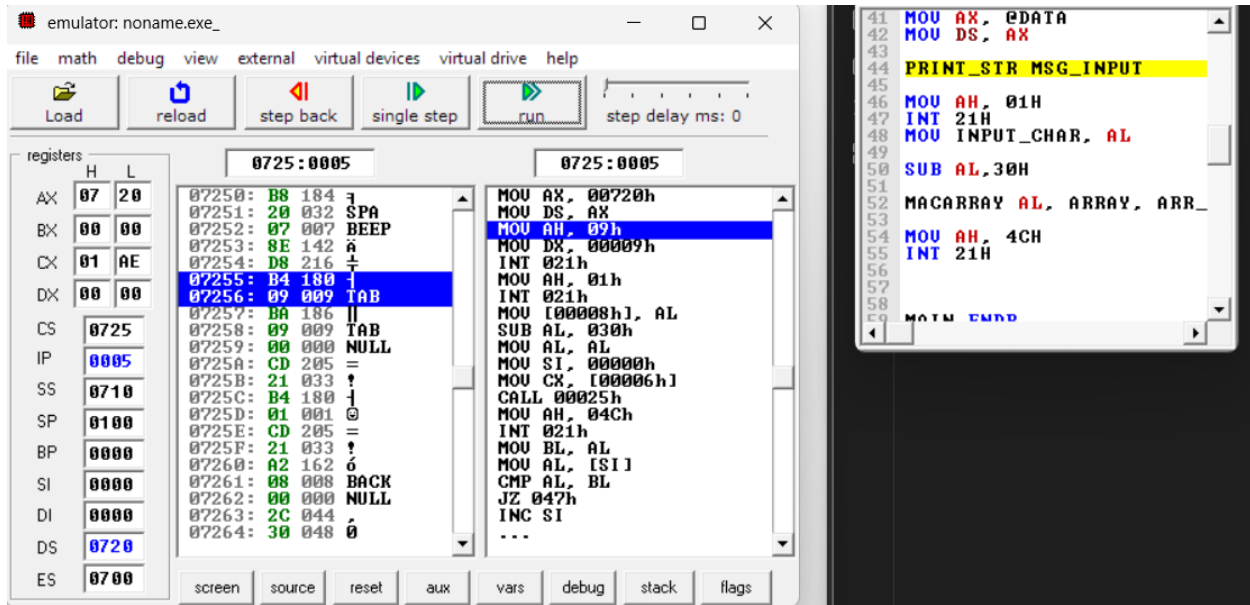
SEARCH_PROC ENDP

END MAIN

Observation:



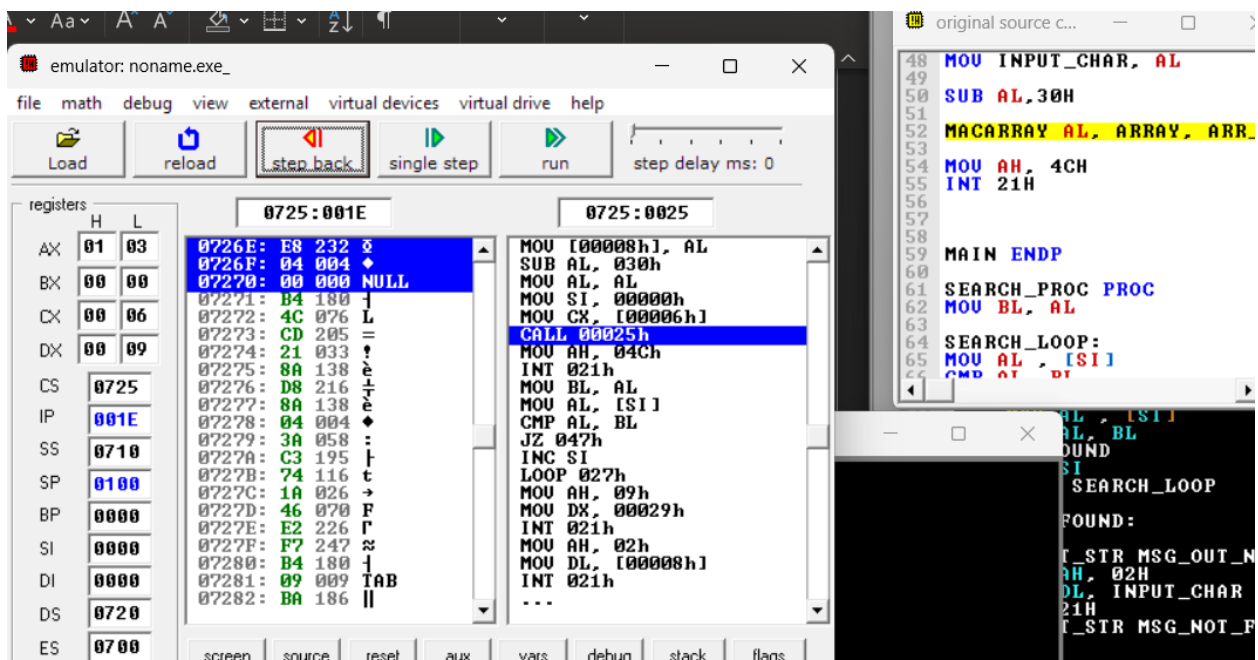
Before executing the macro code (main proc)



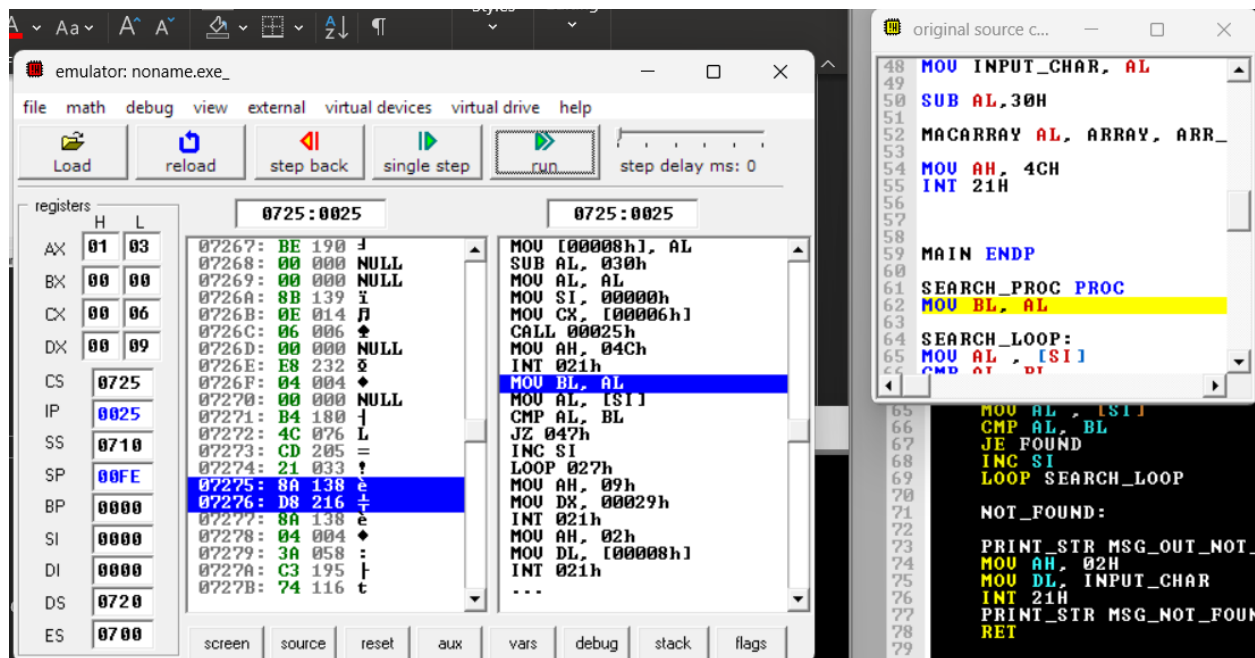
After entering the macro code

As we can see, the macro code are added inline, so the IP moves from only 0003 to 0005, which is the next instruction

ii) Procedure



Before entering the SEARCH_PROC procedure, IP is at 001E



After entering the procedure code area, we jump to a location outside the main proc address range. The IP is 0025

A. Observation of the MACRO

- **Expansion:** In the emulator's "Listing" or "Memory" view, you will notice that the MACARRAY keyword disappears. It is replaced by the actual MOV and LEA instructions.
- **Memory Usage:** If you call the macro twice, you will see those same MOV and LEA instructions duplicated in memory at two different addresses.
- **Parameters:** You will see that the "Dummy Parameters" (digit, address) are replaced by the actual values or offsets you provided in the MAIN proc.

B. Observation of the PROCEDURE

- **Call/Return:** When the instruction pointer (IP) reaches CALL SEARCH_PROC, the emulator pushes the return address to the stack and jumps to a single, specific memory location where the search logic resides.

- **Singularity:** Unlike the macro, the code inside SEARCH_PROC appears only once in the memory map, regardless of how many times it is called.

In this lab task, the **Macro** was used as a convenient "interface" to pass the array address and values into registers. The **Procedure** was used to handle the actual loop logic to save memory, as it prevents the duplication of the search code every time a search is required